

Article

Pest Detection Based on Lightweight Locality-Aware Faster R-CNN

Kai-Run Li ^{1,†}, Li-Jun Duan ^{1,†}, Yang-Jun Deng ^{1,*} , Jin-Ling Liu ², Chen-Feng Long ¹  and Xing-Hui Zhu ^{1,*}

¹ College of Information and Intelligence, Hunan Agricultural University, Changsha 410127, China; viperl1@stu.hunau.edu.cn (K.-R.L.); duanlijunagr@163.com (L.-J.D.); elong@hunau.edu.cn (C.-F.L.)

² College of Agronomy, Hunan Agricultural University, Changsha 410127, China; liujinling@hunau.edu.cn

* Correspondence: dengyangjun@hunau.edu.cn (Y.-J.D.); zhuxh@hunau.net (X.-H.Z.)

† These authors contributed equally to this work.

Abstract: Accurate and timely monitoring of pests is an effective way to minimize the negative effects of pests in agriculture. Since deep learning-based methods have achieved good performance in object detection, they have been successfully applied for pest detection and monitoring. However, the current pest detection methods fail to balance the relationship between computational cost and model accuracy. Therefore, this paper proposes a lightweight, locality-aware faster R-CNN (LLA-RCNN) method for effective pest detection and real-time monitoring. The proposed model uses MobileNetV3 to replace the original backbone, reduce the computational complexity, and compress the size of the model to speed up pest detection. The coordinate attention (CA) blocks are utilized to enhance the locality information for highlighting the objects under complex backgrounds. Furthermore, the generalized intersection over union (GIoU) loss function and region of interest align (RoI Align) technology are used to improve pest detection accuracy. The experimental results on different types of datasets validate that the proposed model not only significantly reduces the number of parameters and floating-point operations (FLOPs), but also achieves better performance than some popular pest detection methods. This demonstrates strong generalization capabilities and provides a feasible method for pest detection on resource-constrained devices.

Keywords: faster R-CNN; lightweight model; MobileNet; pest detection



Citation: Li, K.-R.; Duan, L.-J.; Deng, Y.-J.; Liu, J.-L.; Long, C.-F.; Zhu, X.-H. Pest Detection Based on Lightweight Locality-Aware Faster R-CNN.

Agronomy **2024**, *14*, 2303. <https://doi.org/10.3390/agronomy14102303>

Academic Editor: Hideyoshi Toyoda

Received: 1 August 2024

Revised: 2 October 2024

Accepted: 3 October 2024

Published: 7 October 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Pests have been one of the main reasons for great losses in food yields around the world every year [1]. Chemical pesticides have long been the primary means of preventing and controlling agricultural pests. Although pesticides can successfully eliminate pests, they also pose great hazards to both the natural environment and human health when the pesticides are overused. Employing a technique that can accurately monitor pests in a timely manner is an effective way to avoid the overuse of pesticides. Thus, the pest detection method, as a key technology of pest monitoring, has been a hot topic in plant protection [2].

Traditionally, plant pests and diseases are identified by on-site agricultural experts or by farmers using their own experiences [3]. This method is time-consuming, labor-intensive, and leads to the blind use of drugs because it causes misjudgment resulting from the subjective factors of the inspectors [4]. Undoubtedly, traditional methods are increasingly becoming inadequate and less capable of meeting the demands of modern pest detection. They even hinder the scalability and automation of pest detection. With the development of computer science, automatic pest detection has attracted interest in various fields, such as entomological science [5], environmental science [6], and agricultural engineering [7].

Currently, machine vision-based pest detection is the most popular approach for automatic pest detection. In particular, the machine vision-based pest detection methods

include two types, i.e., shallow machine learning and deep learning. The shallow machine techniques, such as support vector machines (SVMs) [8], linear regression [9], tensor learning [10,11], and manifold learning [12], have been widely applied to pest image classification and identification. For example, Ebrahimi et al. [13] successfully detected thrips in the canopies of strawberry plants using SVMs. Xiao et al. [14] proposed a pest identification solution based on the bag-of-words model and SVMs that successfully identified four common vegetable pests in southern China. To identify stored-grain pests, Qiu et al. [15] developed an intelligent detection system based on image subtraction and self-adaptive enhancement techniques. It is evident from the aforementioned cases that shallow machine learning methods struggle to counteract input disturbances such as rotation, scaling, translation, and viewpoint changes.

Deep learning can automatically extract key features from data without the need for human design, which enables deep learning to deal with data under complex scenes. Moreover, due to the deep stacked structure, deep learning has better data fitting capabilities than shallow machine learning methods. In the end, the deep learning methods, such as RCNN [16], YOLO [17], SSD [18], RetinaNet [19], and so on, have been introduced into the field of automatic pest detection and achieved remarkable accuracy. Specifically, Xia et al. [20] proposed a CNN-based crop pest multi-classification model. This model significantly outperformed traditional insect classification algorithms in terms of detection accuracy. Sabanci et al. [21] proposed a convolutional recursive mixture network that combines AlexNet and Bi-directional long short-term memory (BiLSTM) for wheat pest detection. Selvaraj et al. [22] used SSD for banana pest detection and achieved faster and more accurate detection of specific pest species. Liu et al. [23] proposed a model called PestNet that combines multiple neural networks for precise pest detection and control. In addition, these models consider characteristics such as small size, protective coloration, and variable morphology of pests, some pest detection methods focus on enhancing the model's ability to distinguish feature vectors to achieve better recognition results. For example, Li et al. [24] used data and temporal augmentation strategies to enhance CNN networks, achieving improved accuracy in wheat pest recognition. Zhang et al. [25] used rotation detection algorithms to improve the ability of the YOLO model to recognize pests in complex backgrounds. Liu et al. [26] and Sun et al. [27] improved YOLOv3 and SSD, respectively, using multi-scale feature fusion. This enabled the models to use low-dimensional features and achieve fast and accurate detection of both tomato pests and corn leaf blight. Additionally, some other models have been enhanced to enhance the feature extraction ability of the backbone. Dai et al. [28] combined the YOLOv5m model with the Swin transformer backbone network, which significantly improved the accuracy of the model in detecting different pests. Tian et al. [29] used DenseNet to optimize the low-resolution feature utilization efficiency of the YOLOv3 model, significantly improving the detection of apple anthracnose lesions. Li et al. [30] proposed the YOLO-JD model by inserting two feature extraction enhancement modules and a pyramid pooling module into YOLOv3, enabling the precise identification of ephedra pests. Although these studies have made significant progress in pest recognition accuracy, their implementation requires substantial computational resources, which makes it difficult to deploy them on small-scale devices with limited computational resources and significantly limits the automation and scalability of real-time pest detection devices.

To address the above issues, a series of lightweight deep learning methods have been proposed and applied for pest detection. Cheng et al. [31] improved recognition capabilities by combining the k-means++ optimization with the lightweight YOLO-Lite model and achieved precise identification of multiple pests while reducing computational complexity. Other cases achieved lightweight results by adjusting the structures of the generic models. Li et al. [32] proposed a method that uses EfficientNet to lighten YOLO and PANet to fuse multi-scale features to enhance accuracy. Additionally, Xiang et al. [33] proposed the YOLO-pest model using the ConvNext module from ViT to improve BottleNeck and achieve lower computational complexity. They also used the CAC3 module to address the

problems with feature extraction in small samples. Yang et al. [34] inserted CSPResNext50 and VoVVGSCSP into YOLOv7, which reduced the computational and network structure complexity while maintaining a high rate of feature reuse. The literature has shown that all these methods, as derived models of the YOLO family, highlight detection speed. The YOLO series of single-stage object detection models is characterized by its lightweight models and fast detection speeds; some of the models can even detect continuously at over 60 frames per second (FPS). However, the lightweight pest detection methods mainly focus on accelerating the speed of detection and ignore the balance between accuracy and speed. For the scenario of pest detection, accuracy should be prioritized over the speed in the order of seconds.

According to recent pieces of literature, two-stage object detection models, such as the R-CNN-based methods, generally outperform the YOLO-based one-stage ones in terms of accuracy under similar conditions. Thus, to balance the accuracy and computational cost, this paper proposes a faster R-CNN-based lightweight pest detection model named LLA-RCNN. The model achieves the effective recognition of various insects with a significant reduction in computational complexity by making reasonable adjustments to its structure. Therefore, the model can be deployed more widely on small-scale devices, providing an effective solution for the scale and automation of pest detection. It also serves as a reference for future research. In summary, the significant contributions of this work are as follows:

1. A lightweight backbone mode MobileNetV3X is integrated into the faster R-CNN framework to efficiently extract pest discriminative features from images, which can significantly reduce the computational complexity and the model size.
2. To enhance the robustness and generalization ability of the model, a CA attention module is introduced into the backbone. This module can encode high-quality position information from different channels into feature vectors, which allows the model to better detect inconspicuous pests in complex backgrounds.
3. Experiments on two different types of pest image data are conducted to verify the efficiency of the proposed method. The associated experimental results demonstrate that LLA-RCNN achieves better pest detection performance than several popular methods, including YOLOv8s, YOLOv5s, CenterNet, and faster R-CNN. Additionally, ablation experiments are conducted to separately analyze the rationality and necessity of each improvement.

2. Materials and Methods

2.1. Data Description

Different datasets were used to validate the model. This study used two datasets, one titled ‘Rapeseed Pests dataset for 5 kinds’ (RP5), which was collected by the authors, and the other titled ‘Forestry Pests dataset for 6 kinds’ (FP6), which is part of the pest detection dataset that is publicly available on the Baidu PaddlePaddle platform.

Most images in the RP5 dataset were collected from the Rapeseed Experimental Base in Hunan Province, China. The remaining portion of the RP5 dataset was extracted from previous studies and websites. All images had resolutions greater than 300 pixels, both vertically and horizontally. Following the removal of anomalous images caused by defocusing, blurring, and overexposure, the remaining 5439 images were used as samples for the RP5 dataset, which included five common rapeseed pests. Detailed information is provided in Table 1.

Table 1. The number and proportion of image samples in the RP5 dataset.

Kinds	Aphid	Caterpillar	Dish	Jumping	Leave
Numbers	1206	1768	917	720	836
Ratio	22.1%	32.5%	16.8%	13.2%	15.3%

The FP6 dataset was based on a collaborative forestry pest control project developed by Baidu and Beijing Forestry University. This dataset can be downloaded from the Baidu PaddlePaddle platform [35]. The FP6 dataset contained 2183 images and 10,832 objects of six common forestry pests. Detailed information is provided in Table 2. To minimize the effect of dataset splitting on the experiments, the dataset was randomly split into training, test, and evaluation sets in a ratio of 8:1:1. Additionally, random operations, such as rotation, translation, flipping, adding salt-and-pepper noise, and mixing, were applied to the training set to improve the robustness of the model.

Table 2. The number and proportion of image samples in the FP6 dataset.

Kinds	Boerner	Leconte	Acuminatus	Armandi	Coleoptera	Linnaeus
Numbers	1848	2661	1126	1930	2222	1045
Ratio	17.1%	24.6%	10.4%	17.8%	20.5%	9.6%

2.2. Framework for the Model

To achieve accurate pest detection on resource-constrained devices, this study improved the existing object detection models in two stages. In the first stage, the full-sized backbone network in the original object detection model was replaced with a smaller lightweight backbone to reduce the model's computation requirement. This change led to a significant reduction in the model size and a small loss in the detection accuracy compared to the original model. In the second stage, to compensate for the loss of detection accuracy in stage one, targeted improvements for both the backbone and downstream functions were implemented. As a two-stage object detection model, faster R-CNN [36] can more accurately identify smaller targets compared to single-stage models such as YOLOv3 [37], YOLOx [38], and CenterNet [39]. Despite the trade-offs in terms of model parameter count and inference speed, we believe that it is more suitable than other models for pest detection tasks. Therefore, the faster R-CNN was selected as the prototype model. The final developed model was a lightweight object detection model based on faster R-CNN, which we refer to as locality-aware lightweight faster R-CNN (LLA-RCNN).

The LLA-RCNN model consisted of four parts: a feature extraction network (MobileNetv3X), a region proposal network (RPN), a region of interest Align (RoI Align) module, and linear regression heads. The modeling process was divided into three steps. First, the backbone extracted feature maps from images and suggested data for downstream tasks. Second, the region proposal network generated proposal boxes based on feature maps; RoI Align then selected some of these proposal boxes based on probabilities and adjusted their sizes to match the original image. Finally, linear regression heads were responsible for regressing the coordinates and categories of the predicted boxes. The structure of the LLA-RCNN model is illustrated in Figure 1.

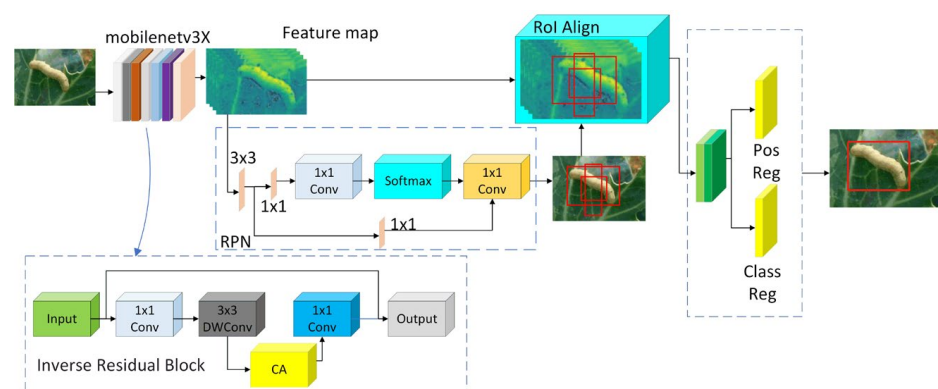


Figure 1. The structure of LLA-RCNN. RPN is the region proposal network. CA is the coordinate attention block.

2.3. Loss Function

The loss function of the LLA-RCNN can be expressed as L_{loss} , which can be described as Equation (1) and can be divided into two components, the classification loss L_{cls} , and the regression loss L_{reg} , sign as L_{loss} , can be described as

$$L_{loss} = L_{cls} + L_{reg} \quad (1)$$

The classification loss L_{cls} can be described as

$$L_{cls} = -\log[p^*p + (1 - p^*)(1 - p)] \quad (2)$$

where p is the predicted probability of the object class and p^* is the ground truth probability of the object class.

The regression loss L_{reg} can be described as

$$L_{reg} = 1 - L_{GIoU} \quad (3)$$

The generalized intersection-over-union (GIoU) loss function [40] is used to estimate the spatial relationship between the predicted bounding boxes and ground truth boxes in the LLA-RCNN model. This is better than the regression loss function smoothL1, which is used in the original faster R-CNN. The GIoU function can accurately measure the overlap between the predicted and ground-truth bounding boxes. In particular, when the predicted and ground-truth bounding boxes do not overlap, the GIoU imposes a penalty value on the loss function. This feature enables the entire model to converge faster and ultimately enhances its accuracy. The formula for the GIoU loss is represented as

$$IoU = \frac{A \cap B}{A \cup B} \quad (4)$$

$$L_{GIoU} = IoU - \frac{C - A \cup B}{C} \quad (5)$$

In Equations (4) and (5), A is the predicted box, B is the ground-truth box, and C is the bounding rectangle that encloses both A and B , as illustrated in Figure 2.



Figure 2. The spatial relationships of the boxes in GIoU.

2.4. MobileNetv3X Lightweight Backbone

After analyzing the parameter sizes of different parts of the faster R-CNN model, it can be concluded that the backbone contributed the most to the overall model parameters. To ensure detection accuracy, faster R-CNN typically uses full-sized CNN models, such as ResNet50 [41] or VGG16 [42], as the backbone to provide high-quality feature maps for downstream tasks. Although these backbones can effectively extract feature information from images, the large number of parameters they require has become a significant obstacle for lightweight faster R-CNNs. Therefore, it appears that a lightweight model based on a lightweight backbone network to replace the full-sized backbone network will achieve a significant reduction in the model size. However, simply replacing the backbone with

a lightweight backbone network results in a sharp decrease in accuracy. Taking the MobileNet series as an example, model size reduction is mainly achieved by using Depthwise Separable Convolution (Dwise Conv) instead of the standard convolutions. By splitting the convolution into a depthwise convolution and a pointwise convolution, Dwise Conv effectively reduces model size and floating-point computation. If the input feature map has dimensions of $D_f \times D_f \times M$ and the convolution kernel has a size of $D_k \times D_k$ with N output channels, the number of parameters and the floating-point computation for a Dwise Conv and a regular convolution can be computed as follows:

$$\frac{\text{Dwise Conv}}{\text{Conv}} = \frac{D_k \cdot D_k \cdot M + M \cdot N}{D_k \cdot D_k \cdot M \cdot N} = \frac{1}{N} + \frac{1}{D_k^2} \quad (6)$$

$$\frac{\text{Dwise Conv}}{\text{Conv}} = \frac{D_k \cdot D_k \cdot M \cdot D_f \cdot D_f + M \cdot N \cdot D_f \cdot D_f}{D_k \cdot D_k \cdot M \cdot N \cdot D_f \cdot D_f} = \frac{1}{N} + \frac{1}{D_k^2} \quad (7)$$

In Equations (6) and (7), when N is large, the term $\frac{1}{N}$ becomes small enough to be negligible. Additionally, for typical cases, D_k is usually 3. Therefore, in practice, the Dwise Conv can reduce the model's size and floating-point computation to approximately $\frac{1}{9}$ of the original values.

Of course, there is a clear downside to this approach. Splitting the convolution into depthwise and pointwise convolutions can disrupt the relationships between spatial and channel features in the feature maps. This separation reduces the ability of the Dwise Conv to capture some of the important correlations between these dimensions. This leads to the inability of the lightweight backbone to provide sufficiently high-quality feature maps for downstream tasks, thereby affecting the recognition accuracy of these downstream tasks.

To address this issue, this study proposes a new backbone network based on MobileNetV3 [43]. To ensure that the backbone paid more attention to small-sized pest targets, coordinate attention (CA) blocks were introduced into its inverted residual blocks to enhance the model's ability to capture features in the local space [44]. Due to the cross-axis computation method from the CA block, this new model is named MobileNetV3X, where 'X' means cross-axis computation. The two types of inverted residual blocks used in MobileNetV3X are shown in Figure 3.

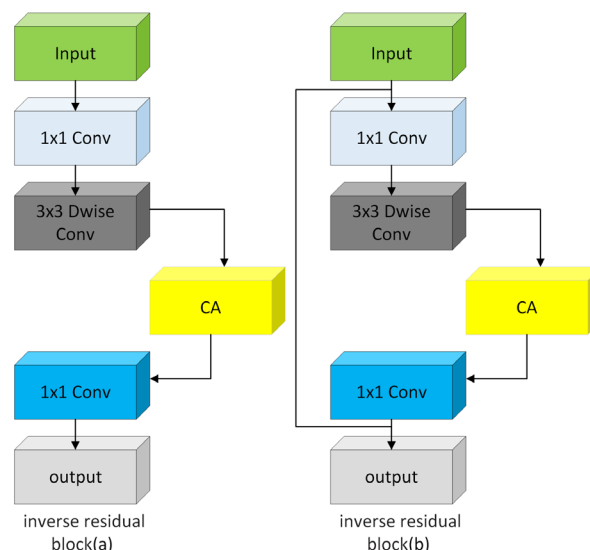


Figure 3. The two types of inverse residual block structure diagrams for MobileNetV3X.

The calculation process of the CA module is illustrated in Figure 4 and can also be expressed as

$$y_c(i, j) = x_c(i, j) \times g_c^h(i) \times g_c^w(j) \quad (8)$$

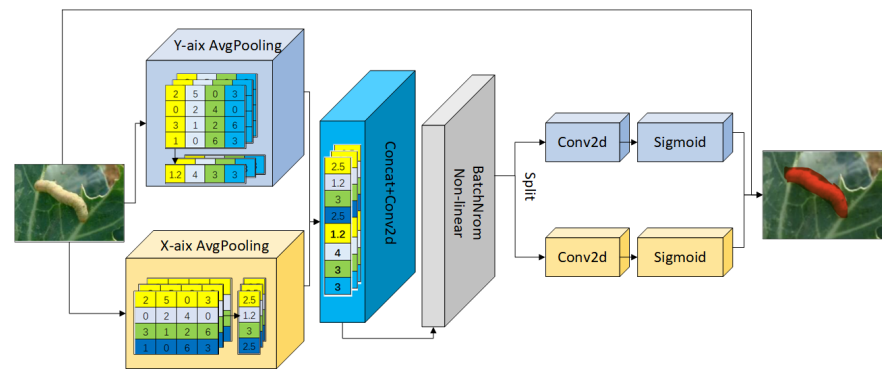


Figure 4. The Coordinate Attention (CA) Module.

In Equation (8), x_c is the original feature map, while g_c^h and g_c^w are the weighting coefficients of the original feature map in the height and width dimensions, respectively. The computational process is outlined as follows:

Step 1: the original feature map is subjected to separate average pooling operations along the x-axis (horizontal) and y-axis (vertical) directions. This yields two independent feature maps, denoted as z^h and z^w , respectively. This process is mathematically described as

$$z_c^h = \frac{1}{W} \sum_{0 \leq i \leq w} |x_c(h, i)| \quad (9)$$

$$z_c^w = \frac{1}{H} \sum_{0 \leq j \leq w} |x_c(j, w)| \quad (10)$$

Step 2: concatenate z^w and z^h and pass them through a shared 1×1 convolutional kernel for dimension reduction. Perform subsequent batch normalization and sequentially apply the rectified linear unit activation function to obtain the feature map f . This process can be mathematically formulated as

$$f = \sigma \left(F_1 \left(\begin{bmatrix} z^h \\ z^w \end{bmatrix} \right) \right) \quad (11)$$

In Equation (11), F_1 represents the dimension reduction operation, which reduces the number of channels to $1/r$ of the original, where r is a hyperparameter and σ is the activation function.

Step 3: Separately split and channel-adjust the feature map along the x-axes and y-axes. Subsequently pass them through a sigmoid activation function to obtain attention weights g^h and g^w in the two directions. This can be mathematically formulated as

$$g^h = \sigma \left(F_h \left(f^h \right) \right) \quad (12)$$

$$g^w = \sigma \left(F_w \left(f^w \right) \right) \quad (13)$$

In Equations (12) and (13), F_h and F_w are channel adjustments and σ is the activation function.

2.5. RPN

The RPN (region proposal network) uses CNNs to generate a large number of anchor boxes at once, significantly reducing the time needed to create anchor boxes as compared to traditional sliding window algorithms such as Selective Search. This in turn reduces the overall recognition time. As shown in Figure 1, the RPN is essentially divided into two branches. The main branch (the top branch) generates 9 different anchor boxes for each feature point with aspect ratios of 1:1, 1:2, and 2:1. It then uses softmax to determine whether the region contains an object. The other branch uses linear regression to refine the anchor boxes, making the suggestions more accurate.

2.6. RoI Align

Due to the changes in the size of the feature map caused by downsampling the CNNs, the anchor boxes provided by the RPN cannot be mapped directly to the feature vectors. Therefore, the model introduces RoI pooling to match the anchor boxes to the feature vectors. For example, an anchor box of 305×305 in the original image is reduced to a size of 9.53×9.53 after four downsampling operations. RoI pooling can only handle integers, so the model rounds this down to 9×9 . If the final feature map processed by the backbone network is 7×7 , this region is further mapped to 1.28×1.28 . Rounding again reduces the final result to 1×1 . As a result of these two rounding operations, the original 305×305 anchor box is mapped to 224×224 , which will undoubtedly have a significant impact on the regression process in the subsequent classification head. In contrast, RoI Align does not require any rounding when performing the same task. The 305×305 anchor box would be mapped directly to 9.53×9.53 and then further mapped to 1.36×1.36 , thereby maintaining spatial accuracy.

After completing the mapping of the anchor box, the operation of the RoI Align can be summarized as follows: First, iterate over each candidate region and divide it into a 7×7 grid of sub-regions. Each sub-region is further divided into four cells. No quantization operations are involved in this division process. Next, use bilinear interpolation [45] to calculate the coordinates of the center pixel for each cell and extract the value at that coordinate as the value for the respective cell. Finally, apply max pooling to aggregate the values of each subregion separately, resulting in the final feature map. Compared to RoI pooling, RoI Align has finer granularity, which helps to avoid the loss of information from the original feature map during processing. In addition, the use of bilinear interpolation prevents the disruption of information integrity caused by quantization operations.

2.7. Presets and Evaluation

In this section, a series of experiments were designed to demonstrate that the LLA-RCNN model can achieve comparable or even better detection accuracy than other models in pest detection tasks, requiring fewer resources. All the experiments were configured according to the preset hyperparameters listed in Table 3. All control experiments used the same dataset and dataset split to ensure the singularity of the experimental variables. The fine-tuning optimization [46] method was applied to all models, which involved freezing the backbone network for 50 epochs of initial training followed by unfreezing for an additional 150 epochs of overall training. The resulting mean average precision (mAP) data from these experiments were used as the experimental results.

Table 3. The preset hyperparameter for training.

Name	Value
Initial learning rate	1×10^{-4}
Input shape	600×600
Momentum	0.9
Confidence threshold	0.5
Batch size	8
Optimizer	Adam
Freeze epoch	50
Un-Freeze epoch	150

To quantify the performance of the model, the evaluation metric used in this article is average precision (AP), which is a metric used to measure the accuracy of object detectors. It assesses the performance of the detector using both recall and precision. The specific calculation method involves selecting the highest precision for each unique recall and then plotting the precision-recall (P-R) curve. The area under the curve represents the AP. The following equations can be used to describe this method:

$$P_{interp}(r) = \max_{\hat{r} > r} p(\hat{r}) \quad (14)$$

$$AP = \sum (r_{n+1} - r_n) P_{interp}(r_{n+1}) \quad (15)$$

$$mAP = \frac{\sum_{i=1}^k AP_i}{k} \quad (16)$$

3. Experiments and Analysis

3.1. Experiments on the Laboratory Backgrounds Dataset

To evaluate the performance of LLA-RCNN, FP6, a public dataset that was collected under laboratory backgrounds, is employed for the simulation experiments. The experimental results are listed in Table 4. In Table 4, the item ‘Time’ represents the time taken to predict each image. The result for this item is obtained by averaging the time taken by the model to predict the entire test set. For the FP6 dataset, regarding accuracy, LLA-RCNN outperformed R-CNN by 3.54%, CenterNet by 2.88%, and YOLOv5s by 7.72%, and fell only 0.5% behind the performance of YOLOv8s. Of all the models, LLA-RCNN has the fewest parameters, being 42% smaller than the second ranked YOLOv8s. However, LLA-RCNN’s speed was increased by 31%, making it comparable to the single-stage CenterNet and only 38% slower than YOLOv8s, which remains the fastest model.

The reason why LLA-RCNN, despite having an advantage in model size, lags behind YOLOv8s in terms of floating-point operations and computation speed is that it has a more complex classification head structure. The head of LLA-RCNN includes two separate fully connected layers to handle position regression and class prediction. The computational method of fully connected layers ensures that they are highly resource-intensive. Through quantitative analysis, the classification head, while accounting for only 22% of the model size, contributes 88% of the model’s floating-point operations and 64% of the computation time. This has become one of the key areas for future improvement. Through quantitative analysis, it was determined that the classification head consumes 88% of the floating-point operation resources and 64% of the computation time, despite accounting for only 22% of the overall model size. This has become one of the key areas for future improvement.

Table 4. The performance of the FP6 dataset compared with other typical models.

Models	mAP (%)	Model Size (M)	Floating-Point Operations (GFLOPS)	Time (s)
YOLOv5s	88.09	<u>46.658</u>	114.845	0.035
CenterNet	92.93	32.665	98.294	0.047
YOLOv8s	96.31	11.138	28.658	0.029
Faster R-CNN	92.27	28.327	940.987	0.069
LLA-RCNN	95.81	<u>6.430</u>	74.571	0.047

Bold and underline are used to mark the best data in each item.

Furthermore, as shown in Figure 5, a set of prediction results for these models was selected from the FP6 dataset. YOLOv5s and CenterNet had varying missed-detection rates while Faster R-CNN misidentified an occluded acuminatus. Although both YOLOv8s and LLA-RCNN correctly identified all pest targets, YOLOv8s produced in-stances where insect body parts, such as prominent antennae or wings, were excluded from the bounding box (as seen with a Leconte on the left side of the image).

To further demonstrate the ability of the model to detect different types of pests, Table 5 shows the AP for each pest category across all models in the comparison. As shown in Table 5, LLA-RCNN achieved the best performance in the Boerner, Leconte, and Armandi categories while also securing second place in the acuminatus and Linnaeus categories. This performance demonstrates that LLA-RCNN, with its minimal model size, efficiently identifies different pest species in the FP6 dataset, highlighting its strong practical value.



Figure 5. The sample images from the FP6 dataset for comparison with other typical models. (a) Ground-truth, (b) YOLOv5s, (c) CenterNet, (d) YOLOv8s, (e) faster R-CNN, and (f) LLA-RCNN.

Table 5. The performance accuracy of the FP6 dataset compared with other typical models.

Models	Boerner (%)	Leconte (%)	Linnaeus (%)	Acuminatus (%)	Armandi (%)	Coleoptera (%)
YOLOv5s	99.21	99.18	82.96	70.87	92.71	83.61
CenterNet	97.79	98.94	93.40	86.55	91.21	89.69
YOLOv8s	99.13	99.31	96.53	91.25	96.61	95.03
Faster R-CNN	99.28	99.36	77.57	87.80	96.96	92.65
LLA-RCNN	99.60	99.81	94.62	90.93	97.53	92.37

Bold and underline are used to mark the best data in each item.

3.2. Experiments on the Natural Backgrounds Dataset

To further demonstrate the effectiveness of the proposed method, a set of control experiments was designed based on the RP5 dataset. The control group included classical models in the field of object detection, such as faster R-CNN and CenterNet. To ensure the objectivity and fairness of the experiments, all experimental conditions, except for the models, were kept consistent. The results are based on the best performance of each algorithm in the experiments, as listed in Table 6. Furthermore, a set of prediction results was selected for these models on the RP5 dataset, as shown in Figure 6.

Table 6 indicates that the LLA-RCNN model reduced the model parameters by 77% and FLOPs by 92% compared with the original faster R-CNN while also maintaining a

comparable AP. Because it is a two-stage algorithm, its accuracy is significantly higher than that of a one-stage algorithm. In terms of accuracy, LLA-RCNN performed 1.21% better as compared with YOLOv5s. Additionally, in terms of computational speed, the simplicity of the RP5 dataset improved the processing speed of all models compared to FP6. This reduced the gap between LLA-RCNN and the fastest model, YOLOv8s, to 29%. At the same time, LLA-RCNN retained the smallest model size, making it highly suitable for use on resource-constrained devices. Figure 6 shows that faster R-CNN and LLA-RCNN provide high levels of confidence in pest identification results.

Table 6. The performance of the RP5 dataset compared with other typical models.

Models	mAP (%)	Model Size (M)	Floating-Point Operations (GFLOP)	Time (s)
YOLOv5s	96.04	46.653	114.627	0.028
CenterNet	97.87	32.664	70.217	0.030
YOLOv8s	97.35	11.138	28.656	0.022
Faster R-CNN	97.71	28.316	940.972	0.042
LLA-RCNN	97.25	6.424	74.563	0.031

Bold and underline are used to mark the best data in each item.



Figure 6. The sample images of the LLA-RCNN-predicted RP5 dataset.

As shown in Table 7, LLA-RCNN achieved the best performance in the caterpillar and leaf categories and ranked second in the aphid category. Combined with its performance on the FP6 dataset, LLA-RCNN shows strong adaptability in identifying different pests in different background environments when specifically trained for such tasks.

Table 7. The performance accuracy of the RP5 dataset as compared with other typical models.

Models	Aphids (%)	Caterpillar (%)	Dish (%)	Jumpings (%)	Leaves (%)
YOLOv5s	96.92	91.68	98.47	93.97	99.16
CenterNet	98.33	97.25	98.98	96.10	98.69
YOLOv8s	98.21	96.84	97.52	97.53	96.65
Faster R-CNN	98.17	96.73	98.77	95.80	99.08
LLA-RCNN	98.23	95.42	99.03	93.68	99.89

Bold and underline are used to mark the best data in each item.

3.3. Ablation Experiment

To validate the effectiveness of the improvements introduced in the LLA-RCNN, ablation experiments were conducted on each of these enhancements. Detailed experiments were conducted on both the RP5 and the FP6 datasets. The experimental results are shown in Tables 8 and 9. As the experimental results above show, although replacing the original backbone network with MobileNetV3 significantly reduces the model's computational complexity, this improvement also leads to a noticeable decline in mAP which is particularly pronounced on datasets like FP6, which consists of dense small objects. MobileNetV3X, an improved type of MobileNetV3 by the introduction of CA attention, achieves an obvious increase in mAP at the same level of computational complexity compared to MobileNetV3. Additionally, both RoI Align and GIoU contribute to varying degrees of mAP improvement. GIoU contributes to a 1.11% improvement in mAP, while RoI Align contributes to a 32.16% improvement in mAP compared to the original MobileNetV3 solution in the experiments on the FP6 dataset. These improvements, respectively, result in 2.22% and 2.53% increases in mAP on the RP5 dataset. If all the improvements are implemented simultaneously in the model, the proposed model in this paper can achieve a 43.1% increase in mAP on the FP6 dataset and a 4.02% increase in mAP on the RP5 dataset compared to the MobileNetV3 model while maintaining the same computational complexity.

In conclusion, the proposed LLA-RCNN model is well designed, effectively leveraging the advantages of each module, significantly reducing model parameters and floating-point computations, and enhancing the efficiency of pest detection.

Table 8. The effects of each module on the model performance on the FP6 dataset.

Backbone	Module	mAP (%)	Parameters (M)	FLOPs (GFLOPS)
ResNet50		92.27	28.327	940.985
MobileNetV3		52.77	5.682	74.438
MobileNetV3	with RoI Align	84.92	5.682	74.438
MobileNetV3	with GIoU	53.88	5.682	74.438
MobileNetV3	with RoI Align and GIoU	86.36	5.682	74.438
MobileNetV3X		58.22	6.430	74.571
MobileNetV3X	with RoI Align and GIoU	95.81	6.430	74.571

Bold and underline are used to mark the best data in each item.

Table 9. The effects of each module on the model performance on the RP5 dataset.

Backbone	Module	mAP (%)	Parameters (M)	FLOPs (GFLOPS)
ResNet50		97.71	28.316	940.972
MobileNetV3		93.23	5.676	74.430
MobileNetV3	with RoI Align	95.76	5.676	74.430
MobileNetV3	with GIoU	95.45	5.676	74.430
MobileNetV3	with RoI Align and GIoU	96.69	5.676	74.430
MobileNetV3X		94.45	6.424	74.563
MobileNetV3X	with RoI Align and GIoU	97.25	6.424	74.563

Bold and underline are used to mark the best data in each item.

4. Conclusions

This paper proposed a lightweight model named LLA-RCNN for pest detection, which outperforms other models in terms of detection accuracy with less computational costs. Compared with some classical models, such as faster R-CNN, YOLOv5s, and CenterNet, the proposed LLA-RCNN significantly reduces the computational cost without any accuracy loss. The experimental results validate that LLA-RCNN achieves a good balance between detection accuracy and computational cost. Additionally, LLA-RCNN can effectively detect various pests under both complex natural and monochrome laboratory backgrounds.

This study has several limitations. Specifically, because of the scarcity of publicly available pest datasets and the difficulty in manually collecting pest data, the types and quantities of pest datasets used in this study are limited. This may cause the model to perform poorly when identifying certain unrecorded pest categories. We intend to expand the dataset in future research and optimize the model for categories with poor detection.

In the future, LLA-RCNN can be applied to smartphones, real-time pest monitoring stations, and automated pesticide spraying equipment for real-time pest assessment. Those

applications can guide farmers in developing scientifically effective control measures, which will help to enhance agricultural production intelligence, increase crop yield and quality, and improve farmers' incomes.

Author Contributions: Data curation, K.-R.L. and J.-L.L.; Formal analysis, C.-F.L.; Funding acquisition, X.-H.Z.; Investigation, J.-L.L.; Methodology, K.-R.L. and Y.-J.D.; Project administration, X.-H.Z.; Validation, Y.-J.D. and C.-F.L.; Visualization, L.-J.D.; Writing—original draft, K.-R.L. and L.-J.D.; Writing—review and editing, Y.-J.D. and X.-H.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the National Natural Science Foundation of China under Grant 62401203 and in part by the Hunan Provincial Natural Science Foundation of China under Grants 2022JJ40189 and 2023JJ40333.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Cai, J.; Xiao, D.; Lv, L.; Ye, Y. An early warning model for vegetable pests based on multidimensional data. *Comput. Electron. Agric.* **2019**, *156*, 217–226. [\[CrossRef\]](#)
2. Jiao, L.; Chen, M.; Wang, X.; Du, X.; Dong, D. Monitoring the number and size of pests based on modulated infrared beam sensing technology. *Precis. Agric.* **2018**, *19*, 1100–1112. [\[CrossRef\]](#)
3. Lippi, M.; Bonucci, N.; Carpio, R.F.; Contarini, M.; Speranza, S.; Gasparri, A. A yolo-based pest detection system for precision agriculture. In Proceedings of the 2021 29th Mediterranean Conference on Control and Automation (MED), IEEE, Puglia, Italy, 22–25 June 2021; pp. 342–347.
4. Kandalkar, G.; Deorankar, A.; Chatur, P. Classification of agricultural pests using dwt and back propagation neural networks. *Int. J. Comput. Sci. Inf. Technol.* **2014**, *5*, 4034–4037.
5. Fedor, P.; Vaňhara, J.; Havel, J.; Malenovský, I.; Spellerberg, I. Artificial intelligence in pest insect monitoring. *Syst. Entomol.* **2009**, *34*, 398–400. [\[CrossRef\]](#)
6. Larios, E.; Deng, H.; Zhang, W.; Sarpola, M.; Yuen, J.; Paasch, R.; Moldenke, A.; Lytle, D.; Correa, S.R.; Mortensen, E.; et al. Automated insect identification through concatenated histograms of local appearance features: Feature vector generation and region detection for deformable objects. *Mach. Vis. Appl.* **2008**, *19*, 105–123. [\[CrossRef\]](#)
7. Gaston, K.J.; O'Neill, M.A. Automated species identification: Why not? *Philosophical Transactions of the Royal Society of London. Philos. Trans. R. Soc. London. Ser. B Biol. Sci.* **2004**, *359*, 655–667.
8. Türkoğlu, M.; Hanbay, D. Plant disease and pest detection using deep learning-based features. *Turk. J. Electr. Eng. Comput. Sci.* **2019**, *27*, 1636–1651. [\[CrossRef\]](#)
9. Zhang, T.; Long, C.F.; Deng, Y.J.; Wang, W.Y.; Tan, S.Q.; Li, H.C. Low-rank preserving embedding regression for robust image feature extraction. *IET Comput. Vis.* **2024**, *18*, 124–140. [\[CrossRef\]](#)
10. Deng, Y.J.; Li, H.C.; Tan, S.Q.; Hou, J.; Du, Q.; Plaza, A. t-Linear tensor subspace learning for robust feature extraction of hyperspectral images. *IEEE Trans. Geosci. Remote Sens.* **2023**, *61*, 5501015. [\[CrossRef\]](#)
11. Deng, Y.J.; Yang, M.L.; Li, H.C.; Long, C.F.; Fang, K.; Du, Q. Feature Dimensionality Reduction with L2, p-Norm-Based Robust Embedding Regression for Classification of Hyperspectral Images. *IEEE Trans. Geosci. Remote Sens.* **2024**, *62*, 5509314. [\[CrossRef\]](#)
12. Long, C.F.; Wen, Z.D.; Deng, Y.J.; Hu, T.; Liu, J.L.; Zhu, X.H. Locality Preserved Selective Projection Learning for Rice Variety Identification Based on Leaf Hyperspectral Characteristics. *Agronomy* **2023**, *13*, 2401. [\[CrossRef\]](#)
13. Ebrahimi, M.; Khoshtaghaza, M.H.; Minaei, S.; Jamshidi, B. Vision-based pest detection based on SVM classification method. *Comput. Electron. Agric.* **2017**, *137*, 52–58. [\[CrossRef\]](#)
14. Xiao, D.; Feng, J.; Lin, T.; Pang, C.; Ye, Y. Classification and recognition scheme for vegetable pests based on the BOF-SVM model. *Int. J. Agric. Biol. Eng.* **2018**, *11*, 190–196. [\[CrossRef\]](#)
15. Cheng, X.; Wu, Y.; Zhang, Y.; Yue, Y. Image recognition of stored grain pests based on deep convolutional neural network. *Chin. Agric. Sci. Bull.* **2018**, *34*, 154–158.
16. Girshick, R.; Donahue, J.; Darrell, T.; Malik, J. Rich feature hierarchies for accurate object detection and semantic segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 23–28 June 2014; pp. 580–587.
17. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788.
18. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C.Y.; Berg, A.C. Ssd: Single shot multibox detector. In Proceedings of the Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016; Proceedings, Part I 14. Springer: Berlin/Heidelberg, Germany, 2016; pp. 21–37.

19. Lin, T.Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 2980–2988.
20. Xia, D.; Chen, P.; Wang, B.; Zhang, J.; Xie, C. Insect detection and classification based on an improved convolutional neural network. *Sensors* **2018**, *18*, 4169. [\[CrossRef\]](#)
21. Sabanci, K.; Aslan, M.F.; Ropelewska, E.; Unlarsen, M.F.; Durdu, A. A novel convolutional-recurrent hybrid network for sunn pest-damaged wheat grain detection. *Food Anal. Methods* **2022**, *15*, 1748–1760. [\[CrossRef\]](#)
22. Selvaraj, M.G.; Vergara, A.; Ruiz, H.; Safari, N.; Elayabalan, S.; Ocimati, W.; Blomme, G. AI-powered banana diseases and pest detection. *Plant Methods* **2019**, *15*, 92. [\[CrossRef\]](#)
23. Liu, L.; Wang, R.; Xie, C.; Yang, P.; Wang, F.; Sudirman, S.; Liu, W. PestNet: An end-to-end deep learning approach for large-scale multi-class pest detection and classification. *IEEE Access* **2019**, *7*, 45301–45312. [\[CrossRef\]](#)
24. Li, R.; Wang, R.; Zhang, J.; Xie, C.; Liu, L.; Wang, F.; Chen, H.; Chen, T.; Hu, H.; Jia, X.; et al. An effective data augmentation strategy for CNN-based pest localization and recognition in the field. *IEEE Access* **2019**, *7*, 160274–160283. [\[CrossRef\]](#)
25. Zhang, W.; Xia, X.; Zhou, G.; Du, J.; Chen, T.; Zhang, Z. Research on the identification and detection of field pests in the complex background based on the rotation detection algorithm. *Front. Plant Sci.* **2022**, *13*, 1011499. [\[CrossRef\]](#)
26. Liu, J.; Wang, X. Tomato diseases and pests detection based on improved Yolo V3 convolutional neural network. *Front. Plant Sci.* **2020**, *11*, 521544. [\[CrossRef\]](#) [\[PubMed\]](#)
27. Sun, J.; Yang, Y.; He, X.; Wu, X. Northern maize leaf blight detection under complex field environment based on deep learning. *IEEE Access* **2020**, *8*, 33679–33688. [\[CrossRef\]](#)
28. Dai, M.; Dorjoy, M.M.H.; Miao, H.; Zhang, S. A new pest detection method based on improved YOLOv5m. *Insects* **2023**, *14*, 54. [\[CrossRef\]](#) [\[PubMed\]](#)
29. Tian, Y.; Yang, G.; Wang, Z.; Li, E.; Liang, Z. Detection of apple lesions in orchards based on deep learning methods of cyclegan and yolov3-dense. *J. Sens.* **2019**, *2019*. [\[CrossRef\]](#)
30. Li, D.; Ahmed, F.; Wu, N.; Sethi, A.I. Yolo-JD: A Deep Learning Network for jute diseases and pests detection from images. *Plants* **2022**, *11*, 937. [\[CrossRef\]](#)
31. Cheng, Z.; Huang, R.; Qian, R.; Dong, W.; Zhu, J.; Liu, M. A lightweight crop pest detection method based on convolutional neural networks. *Appl. Sci.* **2022**, *12*, 7378. [\[CrossRef\]](#)
32. Li, K.; Zhu, J.; Li, N. Lightweight automatic identification and location detection model of farmland pests. *Wirel. Commun. Mob. Comput.* **2021**, *2021*, 9937038. [\[CrossRef\]](#)
33. Xiang, Q.; Huang, X.; Huang, Z.; Chen, X.; Cheng, J.; Tang, X. Yolo-pest: An insect pest object detection algorithm via CAC3 module. *Sensors* **2023**, *23*, 3221. [\[CrossRef\]](#)
34. Yang, S.; Xing, Z.; Wang, H.; Dong, X.; Gao, X.; Liu, Z.; Zhang, X.; Li, S.; Zhao, Y. Maize-YOLO: A new high-precision and real-time method for maize pest detection. *Insects* **2023**, *14*, 278. [\[CrossRef\]](#)
35. Baidu. FP6 Dataset. Available online: <https://aistudio.baidu.com/datasetdetail/73985/0> (accessed on 1 October 2024).
36. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. In Proceedings of the Advances in Neural Information Processing Systems 28, Montreal, QC, Canada, 7–12 December 2015.
37. Redmon, J.; Farhadi, A. Yolo3: An incremental improvement. *arXiv* **2018**, arXiv:1804.02767.
38. Ge, Z.; Liu, S.; Wang, F.; Li, Z.; Sun, J. Yolox: Exceeding yolo series in 2021. *arXiv* **2021**, arXiv:2107.08430.
39. Duan, K.; Bai, S.; Xie, L.; Qi, H.; Huang, Q.; Tian, Q. Centernet: Keypoint triplets for object detection. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 6569–6578.
40. Rezatofighi, H.; Tsoi, N.; Gwak, J.; Sadeghian, A.; Reid, I.; Savarese, S. Generalized intersection over union: A metric and a loss for bounding box regression. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 15–20 June 2019; pp. 658–666.
41. He, K.; Zhang, X.; Ren, S.; Sun, J. Identity mappings in deep residual networks. In Proceedings of the Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016; Proceedings, Part IV 14. Springer: Berlin/Heidelberg, Germany, 2016; pp. 630–645.
42. Simonyan, K.; Zisserman, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv* **2014**, arXiv:1409.1556.
43. Howard, A.; Sandler, M.; Chu, G.; Chen, L.C.; Chen, B.; Tan, M.; Wang, W.; Zhu, Y.; Pang, R.; Vasudevan, V.; et al. Searching for mobilenetv3. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 1314–1324.
44. Hou, Q.; Zhou, D.; Feng, J. Coordinate attention for efficient mobile network design. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Nashville, TN, USA, 20–25 June 2021; pp. 13713–13722.
45. He, K.; Gkioxari, G.; Dollár, P.; Girshick, R. Mask r-cnn. In Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 2961–2969.
46. Tajbakhsh, N.; Shin, J.Y.; Gurudu, S.R.; Hurst, R.T.; Kendall, C.B.; Gotway, M.B.; Liang, J. Convolutional neural networks for medical image analysis: Full training or fine tuning? *IEEE Trans. Med. Imaging* **2016**, *35*, 1299–1312. [\[CrossRef\]](#) [\[PubMed\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.